Notebook: Naive_bayes/naive_bayes_from_scratch.ipynb

[--- MARKDOWN CELL ---]

Naive Bayes (Gaussian) from scratch
This notebook implements Gaussian Naive Bayes from scratch. Each cell defines a single function. We use matplotlib for plotting and a separate module for accuracy calculations.

[--- CODE CELL ---]

```
# imports
import csv
import math
import random
from collections import defaultdict
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
# ensure inline plotting when opened in Jupyter viewers
%matplotlib inline
```

[--- CODE CELL ---]

```
def load_csv(path):
    """Load a CSV into a pandas DataFrame. Returns DataFrame.
    path: filesystem path to CSV
    """
    return pd.read_csv(path)
```

[--- CODE CELL ---]

```
def preprocess_df(df, label_column=None):
    """Convert non-numeric columns to numeric where possible and separate X, y.
    If label_column is None, the last column is treated as label.
    Returns X (ndarray), y (list of labels), feature_names (list)
    """
    df2 = df.copy()
    # If label_column not given, assume last column
    if label_column is None:
        label_column = df2.columns[-1]
    # Encode categorical object columns (except label if categorical)
    for col in df2.select_dtypes(include=['object', 'category']).columns:
        if col == label_column:
            df2[col] = df2[col].astype('category').cat.codes
        else:
            df2[col] = pd.to_numeric(df2[col], errors='coerce')
    # Drop rows with NaNs introduced by coercion
    df2 = df2.dropna().reset_index(drop=True)
    X = df2.drop(columns=[label_column]).values.astype(float)
    y = df2[label_column].values.tolist()
    feature_names = list(df2.drop(columns=[label_column]).columns)
    return X, y, feature_names
```

[--- CODE CELL ---]

```
def train_test_split_from_scratch(X, y, test_size=0.3, seed=None):
    """Split X, y into train/test without sklearn. Returns X_train, X_test, y_train, y_test.
    """
    if seed is not None:
        random.seed(seed)
    indices = list(range(len(X)))
    random.shuffle(indices)
    split_at = int(len(indices) * (1 - test_size))
    train_idx = indices[:split_at]
    test_idx = indices[split_at:]
```

```
    X_train = X[train_idx]
    X_test = X[test_idx]
    y_train = [y[i] for i in train_idx]
    y_test = [y[i] for i in test_idx]
    return X_train, X_test, y_train, y_test
```

[--- CODE CELL ---]

```
def summarize_by_class(X, y):
    """For each class, compute mean, stdev and count for each feature.
    Returns dict[class] = list of (mean, stdev, count) for each feature.
    """
    separated = defaultdict(list)
    for xi, label in zip(X, y):
        separated[label].append(xi)
    summaries = {}
    for label, rows in separated.items():
        arr = np.vstack(rows)
        means = np.mean(arr, axis=0)
        stdevs = np.std(arr, axis=0, ddof=1)  # sample stdev
        counts = arr.shape[0] * np.ones(arr.shape[1], dtype=int)
        summaries[label] = list(zip(means.tolist(), stdevs.tolist(), counts.tolist()))
    return summaries
```

[--- CODE CELL ---]

```
def gaussian_probability(x, mean, stdev):
    """Calculate Gaussian probability density for x.
    """
    if stdev == 0:
        return 1.0 if x == mean else 1e-9
    exponent = math.exp(-((x-mean)**2 / (2 * (stdev**2))))
    return (1 / (math.sqrt(2 * math.pi) * stdev)) * exponent
```

[--- CODE CELL ---]

```
def calculate_class_probabilities(summaries, X_row, class_priors=None):
    """Calculate posterior probabilities for each class for a single row.
    summaries: output of summarize_by_class
    class_priors: optional dict of prior probabilities per class
    Returns dict[class] = probability (unnormalized).
    """
    total_probs = {}
    for class_value, feature_summaries in summaries.items():
        # start with prior
        if class_priors and class_value in class_priors:
            total = class_priors[class_value]
        else:
            total = 1.0
        for i, (mean, stdev, _) in enumerate(feature_summaries):
            x = X_row[i]
            prob = gaussian_probability(x, mean, stdev)
            total *= prob
        total_probs[class_value] = total
    return total_probs
```

[--- CODE CELL ---]

```
def predict(summaries, X_test, class_priors=None):
    """Predict class labels for X_test rows.
    Returns list of predictions.
    """
    preds = []
    for row in X_test:
        probs = calculate_class_probabilities(summaries, row, class_priors)
```

```
        # choose class with highest probability
        best_label = max(probs, key=probs.get)
        preds.append(best_label)
    return preds
```

[--- CODE CELL ---]

```
def plot_two_features(X, y, feature_names=None, feat_idx=(0,1), title='Data by class'):
    """Scatter plot of two features colored by class.
    """
    x_idx, y_idx = feat_idx
    plt.figure(figsize=(7,5))
    for label in np.unique(y):
        mask = np.array(y) == label
        plt.scatter(X[mask, x_idx], X[mask, y_idx], label=str(label), alpha=0.7)
    plt.xlabel(feature_names[x_idx] if feature_names else f'feat_{x_idx}')
    plt.ylabel(feature_names[y_idx] if feature_names else f'feat_{y_idx}')
    plt.title(title)
    plt.legend()
    plt.grid(False)
    plt.show()
```

[--- CODE CELL ---]

```
# Runner: load data, train, predict, evaluate and plot
from Naive_bayes import accuracy_scores
# Adjust the path to the CSV if needed; dataset located at workspace EDA/EDA_breast_cancer_dataset/data.csv
csv_path = '../EDA/EDA_breast_cancer_dataset/data.csv'
df = load_csv(csv_path)
# assume the last column is the label column
X, y, feature_names = preprocess_df(df)
X_train, X_test, y_train, y_test = train_test_split_from_scratch(X, y, test_size=0.3, seed=1)
summaries = summarize_by_class(X_train, y_train)
y_pred = predict(summaries, X_test)
acc = accuracy_scores.accuracy_score(y_test, y_pred)
print(f'Accuracy: {acc:.4f}')
# Quick plot of first two features
plot_two_features(X_train, y_train, feature_names=feature_names, feat_idx=(0,1), title='Train data (first two features)')
```